

A Project Report
on
**Design and Verification of a Parameterized 2D Systolic
Array Based
AI Matrix Multiplication Accelerator
Using Verilog, SystemVerilog and UVM**
Submitted for Partial Fulfillment of Award f
**POST GRADUATE CERTIFICATE PROGRAM IN VLSI
DESIGN**

SESSION – 2026-27

Submitted By

Ankit Kumar (260220533010)
Anurag Chauhan (260220533006)



Under the Guidance of

Dr. Ravi Payal
Joint Director



CERTIFICATE

This is to certify that the project entitled "Design and Verification of a Parameterized 2D Systolic Array Based AI Matrix Multiplication Accelerator Using Verilog, SystemVerilog and UVM" submitted by Ankit Kumar and Anurag Chauhan in partial fulfillment of the requirements for the award of the Post Graduate certificate program in VLSI Design at C-DAC Noida is a bonafide record of work carried out under my supervision during the academic year 2026.

Date: __/____/____

Place: Noida

Dr. Ravi Payal
Joint Director
C-DAC Noida

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. Ravi Payal, Joint Director, C-DAC Noida, for his invaluable guidance, encouragement and continuous support throughout this project. We thank the faculty members, technical staff and our colleagues for providing an excellent learning environment. We are also grateful to our families for their constant motivation and support. Throughout the course of this project, we had the opportunity to learn industry-standard RTL design and functional verification methodologies. The valuable technical discussions, timely feedback, and constructive suggestions provided by our guide greatly enhanced our understanding of Verilog HDL, SystemVerilog, Universal Verification Methodology (UVM), and digital design principles. We also acknowledge the support of the Electronics Department at C-DAC Noida for providing excellent laboratory facilities, EDA tools, and a collaborative learning environment. Finally, we sincerely thank our parents and family members for their constant encouragement, patience, and unwavering support throughout this project.

ABSTRACT

Artificial Intelligence applications rely heavily on matrix multiplication operations. This project presents the design and verification of a parameterized two-dimensional systolic array based AI matrix multiplication accelerator using Verilog HDL. A reusable RTL architecture consisting of interconnected Processing Elements performing Multiply-Accumulate operations has been developed. Verification has been carried out using both SystemVerilog and Universal Verification Methodology (UVM), incorporating assertions, functional coverage, constrained-random testing and scoreboarding. The design demonstrates scalable, reusable and synthesizable architecture suitable for FPGA and ASIC implementation.

TABLE OF CONTENTS

• 1. Introduction	8
• 2. Literature Survey	10
• 3. Background	12
• 4. Proposed Architecture	19
• 5. RTL Design	31
• 6. SystemVerilog Verification	47
• 7. UVM Verification	62
• 8. Simulation Results	77
• 9. Applications and conclusions	95
• 10. References	101

LIST OF FIGURES

- Overall Architecture
- Processing Element
- 3×3 Systolic Array
- RTL Flow
- SystemVerilog Flow
- UVM Architecture

LIST OF TABLES

- Research Comparison
- Design Parameters
- Verification Components
- Test Cases

CHAPTER 1

INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) have transformed modern computing by enabling systems to learn from data and perform complex tasks with high accuracy. At the core of almost every deep learning algorithm lies matrix multiplication, which is executed repeatedly during both training and inference. As model sizes continue to grow, conventional processors struggle to meet the computational requirements while maintaining acceptable power consumption. Consequently, dedicated hardware accelerators have become essential for improving computational throughput and energy efficiency.

Among various hardware architectures, the systolic array has emerged as one of the most efficient solutions for matrix multiplication. It consists of a regular grid of Processing Elements (PEs) that execute Multiply–Accumulate (MAC) operations while forwarding operands to neighbouring PEs. This localized communication minimizes memory bandwidth requirements and enables highly parallel computation. Modern AI processors, including Google's Tensor Processing Unit (TPU), employ systolic arrays because of their scalability, high throughput, and efficient data reuse.

1.2 Need for AI Accelerators

The computational demand of neural networks has increased dramatically over the last decade. CPUs offer excellent programmability but execute arithmetic operations sequentially. GPUs improve throughput through massive parallelism but consume significant power and require complex memory hierarchies. Dedicated AI accelerators overcome these limitations by implementing matrix multiplication directly in hardware. Systolic arrays maximize hardware utilization, reduce latency, and provide predictable timing suitable for FPGA and ASIC implementation.

1.3 Objectives of the Project

- Design a parameterized 2D systolic array using Verilog HDL.

- Develop reusable Processing Elements performing Multiply–Accumulate operations.
- Implement scalable RTL using generate constructs.
- Verify the RTL using SystemVerilog and Universal Verification Methodology (UVM).
- Perform assertion-based verification and functional coverage.
- Demonstrate correctness using directed and constrained-random test cases.

1.4 Scope of the Project

The scope of this project includes RTL design of a parameterized systolic array, development of SystemVerilog and UVM verification environments, implementation of assertions and functional coverage, and validation using simulation. The design is intended to be synthesizable and reusable for future FPGA and ASIC implementations.

1.5 Organization of the Report

Chapter 2 presents the literature survey. Chapter 3 explains the theoretical background. Chapter 4 discusses the proposed architecture. Chapter 5 describes the RTL implementation. Chapter 6 presents the SystemVerilog verification environment, while Chapter 7 discusses UVM verification. Chapter 8 summarizes the simulation results, followed by conclusions and future work.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Matrix multiplication is the computational kernel of artificial intelligence accelerators. Numerous researchers have proposed FPGA- and ASIC-based systolic array architectures to improve throughput, scalability and energy efficiency. This chapter reviews the major contributions relevant to the proposed parameterized systolic array accelerator and identifies the research gap addressed by this work.

2.2 Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication (2011)

Presented a Verilog-based FPGA implementation of a systolic array demonstrating that combining pipelining with parallel processing significantly improves matrix multiplication speed. The work established the Processing Element (PE) as the fundamental computational block but primarily focused on RTL implementation and FPGA evaluation.

2.3 Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication (2012)

Improved the implementation methodology and emphasized efficient FPGA realization using Verilog HDL. The architecture highlighted local communication between neighbouring PEs to reduce routing delay and improve throughput.

2.4 Design and Implementation of a Highly Parallel Systolic Array on FPGA (2024)

Introduced a scalable systolic array architecture targeting Vision Transformer acceleration. The design employed highly parallel processing, optimized dataflow and configurable array dimensions for AI workloads.

2.5 Comparative Analysis

Paper	Architecture	Parameterized	Verification	Limitation
2011	2D SA	No	Directed TB	No reusable verification

2012	FPGA SA	No	Directed TB	Limited scalability
2024	Scalable SA	Yes	Functional	No UVM verification
Proposed	Parameterized SA	Yes	SV + UVM	Reusable & scalable

2.6 Research Gap

Existing academic implementations primarily emphasize RTL design and FPGA implementation. Very few provide a complete verification methodology based on SystemVerilog and Universal Verification Methodology (UVM). Assertion-based verification, functional coverage and reusable transaction-level verification environments are generally absent. Furthermore, many implementations are fixed to a specific matrix dimension and cannot be reused without significant RTL modifications.

2.7 Motivation of the Proposed Work

The proposed work addresses these limitations by developing a parameterized 2D systolic array using synthesizable Verilog HDL together with comprehensive SystemVerilog and UVM verification environments. The design supports reusable Processing Elements, configurable array dimensions, assertion-based verification, constrained-random stimulus, functional coverage and automated scoreboarding. These features make the design suitable for both academic learning and industrial verification flows.

2.8 Chapter Summary

This chapter reviewed representative systolic array architectures and AI accelerator implementations. Based on the identified research gap, the proposed design focuses on both reusable RTL architecture and industry-standard verification methodology, providing a complete design-and-verification solution.

CHAPTER3

BACKGROUND

3.1 Introduction

Artificial Intelligence (AI) has transformed modern computing by enabling machines to perform intelligent tasks such as image recognition, speech processing, natural language understanding and autonomous decision making. The rapid growth of deep learning has significantly increased the computational demand of these applications because billions of arithmetic operations are executed during model training and inference. Conventional processors provide flexibility but cannot efficiently handle the massive parallel computations required by modern AI systems. Consequently, dedicated hardware accelerators have become essential. Among these, systolic arrays provide an excellent balance of throughput, scalability and energy efficiency through pipelined execution and localized communication. This project focuses on the design and verification of a parameterized two-dimensional systolic array implemented using Verilog HDL and verified using SystemVerilog and Universal Verification Methodology (UVM).

3.2 Matrix Multiplication

Matrix multiplication is one of the most fundamental mathematical operations in computer science, scientific computing, digital signal processing, and artificial intelligence. In deep learning applications, every convolution layer, fully connected layer, transformer attention mechanism, recurrent neural network, and embedding operation ultimately performs matrix multiplication. Consequently, the efficiency of matrix multiplication directly determines the performance of AI accelerators.

Consider two matrices:

Matrix A of size $m \times k$

and

Matrix B of size $k \times n$

The product of these matrices results in an output matrix C of size $m \times n$, where each element of the output matrix is calculated as:

Equation (3.1)

$$C(i,j) = \sum_{k=1}^n A(i,k) \times B(k,j)$$

where:

- $A(i,k)$ represents the element located at the i th row and k th column of matrix A .
- $B(k,j)$ represents the element located at the k th row and j th column of matrix B .
- $C(i,j)$ represents the element located at the i th row and j th column of the output matrix C .

Each element of the output matrix is therefore computed as the dot product between a row of matrix A and a column of matrix B . This computation involves repeated multiplication followed by accumulation, making the Multiply–Accumulate (MAC) operation the fundamental building block of AI hardware accelerators.

For example, consider the multiplication of two 3×3 matrices.

Matrix A

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Matrix B

$$B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

The resulting output matrix is

$$C = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix}$$

where each element is calculated independently.

The first element of the output matrix is computed as

Equation (3.2)

$$C_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}$$

Similarly,

Equation (3.3)

$$C_{12} = a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32}$$

Equation (3.4)

$$C_{13} = a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33}$$

Likewise,

$$C_{21} = a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31}$$

$$C_{22} = a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32}$$

$$C_{23} = a_{21}b_{13} + a_{22}b_{23} + a_{23}b_{33}$$

$$C_{31} = a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31}$$

$$C_{32} = a_{31}b_{12} + a_{32}b_{22} + a_{33}b_{32}$$

$$C_{33} = a_{31}b_{13} + a_{32}b_{23} + a_{33}b_{33}$$

Thus, every element of the output matrix is obtained by multiplying the corresponding elements of one row of matrix A with one column of matrix B, followed by summing the products. The same procedure is repeated until all elements of the output matrix have been computed.

Although the matrix multiplication algorithm is mathematically straightforward, the number of arithmetic operations increases rapidly as the matrix dimensions become larger. For example, multiplying two 128×128 matrices requires more than 2 million multiplication operations. Modern deep learning models execute billions of such operations during inference, making software-based implementations computationally expensive. Consequently, dedicated hardware architectures such as systolic arrays are employed to perform matrix multiplication with high throughput, low latency, and improved energy efficiency.

3.3 Computational Complexity

For an $N \times N$ matrix multiplication, the total number of multiplication operations is N^3 while the number of addition operations is $N^3 - N^2$. As matrix dimensions increase, computational complexity grows rapidly, motivating the use of dedicated AI hardware accelerators.

Matrix Size Multiplications Additions

3×3	27	18
4×4	64	48
8×8	512	448
16×16	4096	3840
32×32	32768	31744
64×64	262144	258048

3.4 AI Workloads

Convolutional Neural Networks, Vision Transformers and Large Language Models perform repeated matrix multiplication. Efficient hardware architectures are therefore essential to achieve high throughput with low power consumption.

3.5 Processing Element (PE)

The Processing Element is the basic computational unit of the proposed systolic array. Each PE receives operands, performs one Multiply–Accumulate operation, stores the partial sum and forwards operands to neighbouring PEs. This localized communication minimizes routing delay and improves scalability.

3.6 Multiply–Accumulate (MAC)

The MAC operation is expressed as $\text{Result} = \text{Result} + (A \times B)$. Each Processing Element contains one MAC unit, enabling simultaneous computation across the systolic array.

3.7 Parallel Processing

All Processing Elements operate concurrently, allowing multiple matrix elements to be computed simultaneously and significantly improving throughput compared to sequential execution.

3.8 Pipelining

Operands move through the array every clock cycle. After the pipeline is filled, useful computation is produced in every cycle, maximizing hardware utilization.

Clock Cycle Activity

Cycle 0	First operands enter array
Cycle 1	Second operands enter array
Cycle 2	Pipeline begins filling
Cycle 3	Partial results generated
Cycle 4	Multiple PEs active simultaneously
Cycle 5	Continuous MAC operations
Cycle 6	Output matrix generation
Cycle 7	Final results available

3.9 Parameterization

The proposed RTL uses configurable parameters such as N, DATA_WIDTH and ACC_WIDTH. This enables the same RTL to generate different systolic array sizes without structural modifications.

Parameter	Description
N	Array Dimension

Parameter	Description
DATA_WIDTH	Width of Input Data
ACC_WIDTH	Width of Accumulator
clear_acc	Clears all accumulated values
rst	Global Reset

3.10 Chapter Summary

This chapter introduced the theoretical background of the proposed AI accelerator, including matrix multiplication, computational complexity, processing elements, MAC operations, parallel processing, pipelining and parameterization. These concepts form the basis for the proposed architecture presented in the next chapter.

CHAPTER 4

PROPOSED ARCHITECTURE

4.1 Introduction

This chapter presents the proposed architecture of the **Parameterized Two-Dimensional Systolic Array Based AI Matrix Multiplication Accelerator** developed in this project. The architecture is designed to perform matrix multiplication efficiently by exploiting parallel processing, pipelined execution, and localized data communication between neighbouring Processing Elements (PEs). The proposed design is fully parameterized, allowing the same RTL implementation to support different matrix dimensions without requiring structural modifications.

The accelerator consists of an array of interconnected Processing Elements arranged in a regular two-dimensional grid. Each Processing Element performs one **Multiply–Accumulate (MAC)** operation during every clock cycle while simultaneously forwarding the input operands to adjacent Processing Elements. This regular data movement minimizes external memory accesses and enables continuous computation after the pipeline is filled.

Unlike conventional sequential processors, where arithmetic operations are executed one after another, the proposed systolic array performs multiple MAC operations simultaneously. This significantly improves computational throughput while maintaining a simple and modular hardware architecture. The proposed design is implemented using synthesizable Verilog HDL and verified using both SystemVerilog and Universal Verification Methodology (UVM), ensuring correctness through directed as well as constrained-random verification.

The architecture has been designed with scalability and reusability as primary objectives. By modifying only the design parameters, the same RTL source code can generate larger

systolic arrays such as 4×4 , 8×8 , or higher dimensions without changing the overall architecture.

4.2 Overall Architecture

The proposed accelerator consists of the following major components:

- Input Interface
- Processing Element (PE) Array
- Interconnection Network
- Control Logic
- Accumulator Registers
- Output Interface

The complete architecture of the proposed system is illustrated in **Figure 4.1**.

Figure 4.1 Overall Architecture of the Proposed Systolic Array

(Insert the professional architecture diagram here. Use the architecture based on your project.)

The input matrices are streamed into the systolic array from two orthogonal directions. Matrix **A** is supplied horizontally from the left side of the array, while Matrix **B** is supplied vertically from the top. Each Processing Element receives one element from Matrix **A** and one element from Matrix **B**, performs the Multiply–Accumulate operation, forwards both operands to neighbouring Processing Elements, and stores the accumulated partial sum.

After all operands have propagated through the array, the accumulated values stored in each Processing Element collectively form the output matrix **C**.

The proposed architecture follows a highly regular structure, making it suitable for FPGA implementation as well as future ASIC realization.

4.3 Major Components of the Proposed Architecture

The complete systolic array architecture consists of several hardware modules that work together to perform matrix multiplication. Each module has a well-defined functionality and contributes to the overall computational efficiency of the accelerator.

4.3.1 Input Interface

The input interface is responsible for supplying matrix elements to the systolic array. Matrix **A** is injected row-wise from the left edge of the array, while Matrix **B** is injected column-wise from the top edge.

The input interface synchronizes the arrival of operands with the system clock, ensuring that corresponding elements reach each Processing Element at the correct time.

The proposed implementation supports configurable input widths through the parameter **DATA_WIDTH**, allowing different operand precisions to be used without modifying the RTL structure.

4.3.2 Processing Element Array

The Processing Element Array is the computational core of the proposed accelerator. It consists of multiple identical Processing Elements connected in a regular two-dimensional mesh.

Each Processing Element performs the following operations during every clock cycle:

1. Receive input operands.
2. Perform multiplication.
3. Add the product to the accumulated result.
4. Store the updated partial sum.

5. Forward both operands to neighbouring Processing Elements.

Because all Processing Elements operate simultaneously, the array achieves a high degree of parallelism.

For the current implementation, the architecture consists of a 3×3 array containing **nine Processing Elements**.

However, due to parameterization, the same RTL can be used to generate larger arrays without modifying the architecture.

4.3.3 Processing Element Architecture

The Processing Element is the smallest computational block of the proposed systolic array. It is designed to execute one Multiply–Accumulate operation every clock cycle while forwarding operands to adjacent Processing Elements.

Each Processing Element consists of:

- Multiplier
- Accumulator Register
- Input Registers
- Output Registers
- Reset Logic
- Clear Accumulator Logic

The internal architecture of the Processing Element is shown in **Figure 4.2**.

Figure 4.2 Processing Element Block Diagram

(Insert the PE block diagram here.)

The Processing Element receives two operands:

- **a_{in}** from the left neighbour.
- **b_{in}** from the upper neighbour.

The operands are multiplied together, and the resulting product is added to the accumulated value stored in the accumulator register. The updated result is then stored and used in subsequent MAC operations.

Simultaneously, the operands are forwarded as **a_{out}** and **b_{out}** to neighbouring Processing Elements, enabling continuous data movement throughout the array.

4.4 Dataflow in the Proposed Architecture

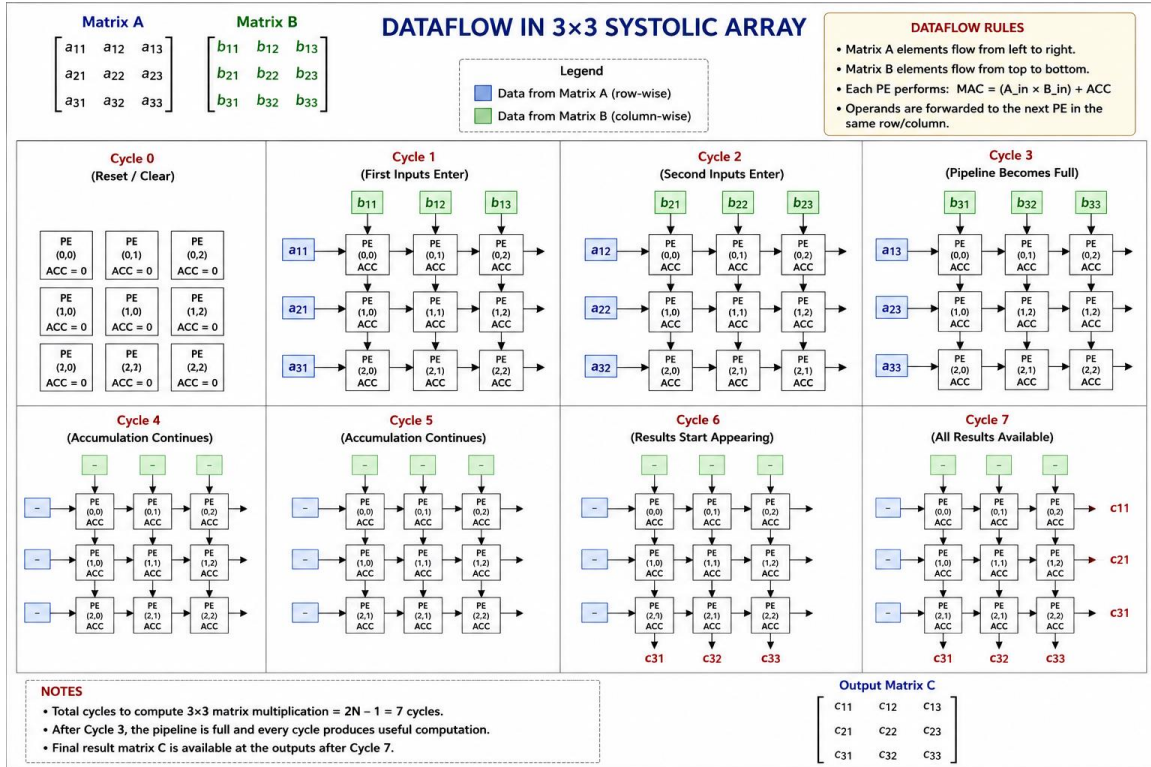
One of the key features of a systolic array is its regular and predictable dataflow. Unlike conventional architectures, where operands are repeatedly fetched from memory, the proposed design allows data to move directly between neighbouring Processing Elements.

The dataflow mechanism can be summarized as follows:

- Matrix **A** propagates horizontally from left to right.
- Matrix **B** propagates vertically from top to bottom.
- Each Processing Element performs one MAC operation whenever valid operands are available.
- Partial sums remain inside the Processing Element until the complete dot product has been computed.
- Final results are collected from the accumulator registers after the pipeline completes.

This localized communication significantly reduces memory traffic, routing complexity, and power consumption while improving computational throughput.

Figure 4.3 Dataflow in the Proposed Systolic Array



4.5 Cycle-by-Cycle Operation of the Proposed Systolic Array

The proposed systolic array performs matrix multiplication through a pipelined sequence of operations. Instead of loading the complete matrices into the processing elements simultaneously, the operands are streamed into the array in a synchronized manner. Matrix **A** is fed horizontally from the left side of the array, whereas Matrix **B** is fed vertically from the top. Each Processing Element (PE) performs one Multiply–Accumulate (MAC) operation whenever both operands become available.

Initially, all accumulator registers are cleared using the **clear_acc** signal. As the first elements of the input matrices enter the array, every Processing Element starts forwarding its received operands to the neighbouring Processing Elements while simultaneously performing MAC operations. This continuous movement of data forms a pipeline, enabling multiple computations to occur concurrently.

The execution of the proposed 3×3 systolic array is completed in seven clock cycles. During the initial clock cycles, the pipeline is filled with input operands. Once the pipeline is full, all Processing Elements perform useful computations during every clock cycle. Finally, after all operands have propagated through the array, the accumulated values stored in each Processing Element represent the elements of the output matrix.

Table 4.1 summarizes the operation of the proposed systolic array during each clock cycle.

Table 4.1 Cycle-by-Cycle Operation

Clock Cycle	Operation
Cycle 0	Reset the array and clear all accumulator registers.
Cycle 1	First row of Matrix A and first column of Matrix B enter the array. Initial MAC operations begin.
Cycle 2	Second operands enter the array. Data propagates to neighbouring Processing Elements while accumulation continues.

Clock Cycle	Operation
Cycle 3	Pipeline becomes fully occupied. All Processing Elements perform MAC operations simultaneously.
Cycle 4	Partial sums continue to accumulate as operands propagate through the array.
Cycle 5	Final multiplication operations are completed in most Processing Elements.
Cycle 6	Output matrix elements begin appearing in accumulator registers.
Cycle 7	Complete output matrix is available for read operation.

The pipelined execution enables multiple matrix elements to be computed simultaneously, thereby improving throughput and reducing overall execution time.

Figure 4.4 should illustrate the movement of matrix elements through the array during each clock cycle. This figure may be developed using the handwritten timing diagram prepared during the design phase.

4.6 RTL Design and Parameterization

The proposed accelerator has been implemented using synthesizable Verilog HDL. The RTL architecture consists of two primary modules:

1. **Processing Element (PE) Module**
2. **Parameterized Systolic Array Module**

The Processing Element performs the basic Multiply–Accumulate operation, whereas the systolic array module interconnects multiple Processing Elements to realize the complete matrix multiplication accelerator.

One of the important features of the proposed design is **parameterization**. Instead of designing a fixed-size systolic array, the RTL has been developed using configurable

parameters that allow different array dimensions and data widths to be generated without modifying the source code.

The major design parameters are listed in Table 4.2.

Table 4.2 Design Parameters

Parameter	Description
N	Dimension of the systolic array
DATA_WIDTH	Width of each input operand
ACC_WIDTH	Width of accumulator register
clk	System clock
rst	Global reset signal
clear_acc	Clears all accumulator registers

For the present implementation, the following parameter values have been used:

- **N = 3**
- **DATA_WIDTH = 8 bits**
- **ACC_WIDTH = 32 bits**

However, the same RTL can be used to implement larger systolic arrays such as **4 × 4**, **8 × 8**, or **16 × 16** by modifying only the parameter **N**.

This parameterized approach improves hardware reusability, reduces design effort, and simplifies future enhancements.

4.7 Control Signals

The operation of the proposed systolic array is controlled using three primary control signals.

Clock Signal (clk)

The clock signal synchronizes all sequential operations performed inside the Processing Elements. Every Multiply–Accumulate operation is executed on the rising edge of the clock. Data propagation between neighbouring Processing Elements also occurs synchronously with the system clock.

Reset Signal (rst)

The reset signal initializes the entire systolic array before the start of computation. When the reset signal is asserted, all registers inside every Processing Element are cleared, ensuring that no residual data affects subsequent computations.

Clear Accumulator Signal (clear_acc)

The **clear_acc** signal is used to clear the accumulator registers while preserving the structural configuration of the array. This feature enables multiple matrix multiplication operations to be executed sequentially without resetting the complete design.

The combination of these control signals provides reliable initialization and synchronization of the proposed accelerator.

4.8 Advantages of the Proposed Architecture

The proposed parameterized systolic array offers several advantages over conventional sequential matrix multiplication architectures.

High Throughput

Since all Processing Elements operate simultaneously, multiple MAC operations are executed during every clock cycle. This significantly increases computational throughput.

Pipelined Execution

Once the pipeline becomes full, every clock cycle contributes useful computation. Consequently, hardware resources remain efficiently utilized throughout the execution.

Localized Communication

Each Processing Element communicates only with its neighbouring Processing Elements. This reduces routing complexity, minimizes communication delay, and lowers power consumption.

Scalable Architecture

The use of Verilog parameters enables the same RTL implementation to support different matrix dimensions. Larger systolic arrays can be generated without modifying the hardware architecture.

Reusable RTL

The modular design approach allows the Processing Element and systolic array modules to be reused in future AI accelerator projects.

FPGA and ASIC Compatibility

The proposed RTL is fully synthesizable and therefore suitable for FPGA prototyping as well as future ASIC implementation.

4.9 Chapter Summary

This chapter presented the proposed architecture of the parameterized two-dimensional systolic array based AI matrix multiplication accelerator. The overall architecture, major hardware components, Processing Element organization, dataflow mechanism, pipelined execution, RTL implementation, parameterization strategy, and control signals were discussed in detail.

The proposed design employs localized communication between neighbouring Processing Elements, enabling high-throughput matrix multiplication with efficient hardware

utilization. The use of configurable design parameters improves scalability and RTL reusability while maintaining a simple modular architecture. The next chapter describes the RTL implementation of the proposed design and explains each Verilog module in detail.

CHAPTER 5

RTL DESIGN AND IMPLEMENTATION

5.1 Introduction

This chapter presents the Register Transfer Level (RTL) implementation of the proposed **Parameterized Two-Dimensional Systolic Array Based AI Matrix Multiplication Accelerator**. The complete design has been developed using synthesizable Verilog Hardware Description Language (HDL). The RTL architecture has been designed with emphasis on modularity, scalability, hardware reusability, and ease of verification.

The proposed implementation consists of two major hardware modules:

1. **Processing Element (PE) Module**
2. **Parameterized Systolic Array Module**

The Processing Element performs the basic Multiply–Accumulate (MAC) operation, whereas the systolic array module interconnects multiple Processing Elements to realize the complete matrix multiplication accelerator.

The RTL has been developed using a parameterized approach, allowing different matrix dimensions and data widths to be generated by simply modifying the design parameters without changing the hardware architecture. This significantly improves the portability and reusability of the design.

5.2 RTL Design Methodology

The complete RTL implementation follows a modular design methodology. Each hardware block is implemented independently and then integrated to form the complete accelerator.

The overall RTL development flow is shown below.

Figure 5.1 RTL Design Flow

Specification

|



RTL Design (Verilog)

|



Processing Element Design

|



Parameterized Systolic Array

|



Simulation



Functional Verification



Synthesis Ready RTL

The modular implementation provides better readability, easier debugging, and improved hardware reuse.

5.3 Processing Element RTL

The Processing Element (PE) is the fundamental computational unit of the proposed architecture. Every matrix multiplication operation performed by the accelerator is

ultimately decomposed into a sequence of Multiply–Accumulate (MAC) operations executed by the individual Processing Elements.

Each Processing Element performs the following operations during every clock cycle:

- Receives one operand from Matrix A.
- Receives one operand from Matrix B.
- Performs multiplication.
- Adds the product to the accumulated partial sum.
- Stores the updated accumulator value.
- Forwards Matrix A data to the right neighbour.
- Forwards Matrix B data to the bottom neighbour.

Since every Processing Element performs identical operations, the architecture becomes highly regular and scalable.

The major ports of the Processing Element are listed in Table 5.1.

Table 5.1 Processing Element Ports

Signal	Direction	Description
clk	Input	System Clock
rst	Input	Global Reset
clear_acc	Input	Clears accumulator
a_in	Input	Matrix A input
b_in	Input	Matrix B input
a_out	Output	Forwarded Matrix A element

Signal	Direction	Description
b_out	Output	Forwarded Matrix B element
result	Output	Accumulated result

The PE receives operands through **a_in** and **b_in**. On every rising edge of the clock, the operands are multiplied and added to the accumulator register. The updated accumulator value is stored and made available through the **result** output.

Simultaneously, the received operands are forwarded to adjacent Processing Elements through **a_out** and **b_out**, enabling continuous data propagation throughout the array.

5.4 Processing Element Internal Operation

Internally, each Processing Element consists of the following hardware blocks:

- Input Registers
- Multiplier
- Adder
- Accumulator Register
- Output Registers
- Reset Logic
- Clear Accumulator Logic

The internal architecture of the PE is illustrated in **Figure 5.2**.

(Insert the PE Block Diagram created earlier.)

The operational sequence of the Processing Element is summarized below:

1. Read input operands.

2. Multiply both operands.
3. Add the product to the previous accumulator value.
4. Store the updated accumulator.
5. Forward operands to neighbouring Processing Elements.
6. Repeat until all matrix elements have propagated through the array.

This pipelined operation allows every Processing Element to execute one MAC operation during each clock cycle.

5.5 Parameterized Processing Element

One of the major advantages of the proposed RTL is parameterization.

Instead of implementing a fixed-width Processing Element, the module uses configurable parameters.

parameter DATA_WIDTH = 8;

parameter ACC_WIDTH = 32;

The **DATA_WIDTH** parameter specifies the width of the input operands, while **ACC_WIDTH** determines the width of the accumulator register.

The accumulator width is selected to prevent overflow during repeated MAC operations.

Using parameters provides several advantages:

- Improved scalability.
- Hardware reusability.
- Easy modification.
- Reduced RTL duplication.

- Better maintainability.

The same Processing Element can therefore be reused in systolic arrays of different dimensions without changing the source code.

5.6 Reset Operation

The proposed Processing Element implements a synchronous reset mechanism.

When the **rst** signal is asserted:

- All accumulator registers are cleared.
- Output registers are initialized.
- Partial sums are reset to zero.
- Operand forwarding registers are initialized.

This ensures that every matrix multiplication begins with a clean hardware state.

The reset logic also simplifies debugging during simulation and verification.

5.7 Clear Accumulator Operation

In addition to the reset signal, the proposed design introduces a dedicated **clear_acc** signal.

Unlike the reset signal, **clear_acc** clears only the accumulator registers while preserving the remaining hardware state.

This enables multiple matrix multiplication operations to be executed sequentially without resetting the complete systolic array.

The operation of **clear_acc** is particularly useful during verification because new test vectors can be applied immediately after clearing the previous accumulated results.

The implementation of this signal also improves hardware flexibility for future AI accelerator applications.

5.8 Chapter Summary

This chapter presented the RTL implementation of the Processing Element used in the proposed parameterized systolic array architecture. The design methodology, module organization, parameterization strategy, internal architecture, reset operation, and accumulator clearing mechanism were discussed in detail. The Processing Element forms the basic computational building block of the complete accelerator and is designed to execute one Multiply–Accumulate operation during every clock cycle while forwarding operands to neighbouring Processing Elements.

The next section explains the implementation of the **Parameterized Systolic Array Module**, including the use of generate statements, interconnection network, signal propagation, and overall RTL organization.

5.9 Parameterized Systolic Array RTL

The parameterized systolic array module represents the top-level hardware block of the proposed AI matrix multiplication accelerator. Its primary function is to instantiate multiple Processing Elements (PEs), establish the required interconnections between neighbouring Processing Elements, and coordinate the movement of matrix elements throughout the array.

Unlike a fixed-size implementation, the proposed RTL employs parameterization to generate systolic arrays of different dimensions using the same Verilog source code. The array size can be modified simply by changing the parameter **N**, thereby eliminating the need to redesign the architecture for different matrix dimensions.

For the present implementation, the array dimension has been selected as **3 × 3**, resulting in a total of **nine Processing Elements**. However, by changing the parameter value, the same RTL can generate larger arrays such as **4 × 4**, **8 × 8**, or even larger accelerator architectures.

The top-level systolic array receives matrix **A** elements from the left side and matrix **B** elements from the top side. Each Processing Element performs one Multiply–Accumulate (MAC) operation and forwards the received operands to adjacent Processing Elements until all computations have been completed.

5.10 Parameterized RTL Declaration

The proposed RTL begins by declaring configurable parameters that define the hardware characteristics of the systolic array.

The important RTL parameters are shown below.

Parameter	Description
N	Number of rows and columns in the systolic array

Parameter	Description
DATA_WIDTH	Width of each input operand
ACC_WIDTH	Width of the accumulator register

The corresponding Verilog declaration is

```
parameter N = 3;
```

```
parameter DATA_WIDTH = 8;
```

```
parameter ACC_WIDTH = 32;
```

The functionality of each parameter is described below.

- **N** determines the size of the systolic array.
- **DATA_WIDTH** specifies the width of every input operand.
- **ACC_WIDTH** specifies the width of the accumulator register used for storing partial sums.

The use of parameters enables the RTL to be scalable and reusable. Increasing the value of **N** automatically increases the number of Processing Elements generated within the array without requiring any modifications to the structural Verilog code.

5.11 Internal Signal Declaration

To facilitate data transfer between neighbouring Processing Elements, internal wire arrays are declared within the systolic array module. These signals connect the outputs of one Processing Element to the inputs of the adjacent Processing Element.

The major internal signals include:

- Horizontal data buses
- Vertical data buses

- Result buses

Horizontal buses carry matrix **A** elements from left to right, whereas vertical buses carry matrix **B** elements from top to bottom.

The accumulator outputs of each Processing Element are connected to the result matrix, which is eventually presented as the output of the accelerator.

The use of dedicated internal buses simplifies routing and ensures organized communication throughout the array.

5.12 Generate Statement

One of the most important features of the proposed RTL is the use of the **generate statement**.

Instead of manually instantiating every Processing Element, the design employs nested generate loops to automatically create the required hardware.

The conceptual implementation is shown below.

```
genvar i, j;
```

```
generate
```

```
  for(i = 0; i < N; i = i + 1)
```

```
    begin : ROW
```

```
      for(j = 0; j < N; j = j + 1)
```

```
        begin : COLUMN
```

```
          pe PE (...);
```

```
        end
```

end

endgenerate

The outer loop creates rows of Processing Elements, while the inner loop creates the columns. Consequently, a total of $N \times N$ **Processing Elements** are generated automatically.

For the current implementation,

$$N = 3$$

Therefore,

$$\text{Number of Processing Elements} = 3 \times 3 = 9$$

The generate statement significantly reduces RTL complexity and improves maintainability.

5.13 Processing Element Interconnection

Each Processing Element communicates only with its immediate neighbours.

The data propagation mechanism is illustrated in **Figure 5.3**.

(Insert 3×3 Processing Element Interconnection Diagram here.)

The communication mechanism operates as follows:

- Matrix **A** propagates horizontally from left to right.
- Matrix **B** propagates vertically from top to bottom.
- Partial sums remain inside each Processing Element.
- Each Processing Element performs one MAC operation every clock cycle.
- Results remain stored until computation is completed.

Since every Processing Element communicates only with adjacent elements, long routing paths are avoided, reducing hardware complexity and improving timing performance.

5.14 Data Propagation

The movement of data inside the proposed systolic array follows a rhythmic and synchronized pattern.

Initially,

- **PE00** receives **A00** and **B00**.
- During the next clock cycle, **PE01** receives the propagated value of **A00**, while **PE10** receives the propagated value of **B00**.
- Similar propagation continues throughout the array until all Processing Elements receive the required operands.

The data movement inside the array can be summarized as follows:

- Matrix **A** flows from **Left** → **Right**
- Matrix **B** flows from **Top** → **Bottom**

The synchronized movement of operands ensures that every Processing Element receives the correct pair of operands required for the corresponding Multiply–Accumulate operation.

This neighbour-to-neighbour communication minimizes memory accesses and enables efficient pipelined execution.

5.15 Result Collection

Each Processing Element maintains its own accumulator register.

During execution, every MAC operation updates the accumulator register with the newly computed partial sum.

The sequence of operations is as follows:

Accumulator → Partial Sum → Updated Partial Sum → Final Matrix Element

Once all operands have propagated through the array, every accumulator contains one element of the output matrix.

For the proposed 3×3 systolic array:

- PE00 generates C00
- PE01 generates C01
- PE02 generates C02
- PE10 generates C10
- PE11 generates C11
- PE12 generates C12
- PE20 generates C20
- PE21 generates C21
- PE22 generates C22

No additional arithmetic operations are required after the pipeline completes because every accumulator already stores the final matrix element.

5.16 Hardware Scalability

The proposed architecture has been designed to support future expansion.

Changing only the parameter

parameter N = 3;

to

parameter N = 4;

automatically generates **16 Processing Elements**.

Similarly,

parameter N = 8;

generates **64 Processing Elements** without modifying any structural RTL.

This demonstrates one of the major strengths of the proposed design—**hardware scalability through parameterization**.

5.17 Advantages of the RTL Implementation

The RTL developed in this project provides several important advantages:

- Fully synthesizable Verilog implementation.
- Modular Processing Element architecture.
- Parameterized systolic array generation.
- Reusable RTL for different array dimensions.
- Simple neighbour-to-neighbour communication.
- High throughput through pipelining.
- Efficient parallel computation.
- Easy verification using SystemVerilog and UVM.

- FPGA and ASIC implementation compatibility.
-

5.18 Chapter Summary

This section presented the implementation of the parameterized systolic array module used in the proposed AI matrix multiplication accelerator. The RTL organization, parameter declaration, internal signal routing, generate statements, Processing Element interconnection, data propagation mechanism, result collection strategy, and scalability features were discussed in detail.

The modular and parameterized implementation enables the same RTL source code to generate systolic arrays of different dimensions while maintaining a regular architecture and efficient neighbour-to-neighbour communication. This approach simplifies hardware development, improves reusability, and provides an efficient platform for high-performance AI computation.

The next chapter presents the verification methodology adopted for the proposed design, including SystemVerilog-based verification followed by Universal Verification Methodology (UVM), functional coverage, assertions, scoreboarding, and constrained-random testing to ensure complete functional correctness.

CHAPTER 6

SYSTEMVERILOG VERIFICATION

6.1 Introduction

Verification is one of the most critical phases of digital integrated circuit design. As the complexity of modern VLSI systems continues to increase, ensuring the functional correctness of hardware designs has become increasingly challenging. Industry reports indicate that more than seventy percent of the total design cycle is devoted to verification activities. Consequently, an efficient verification methodology is essential to ensure that the implemented hardware behaves exactly according to its specification.

In this project, the proposed parameterized two-dimensional systolic array based AI matrix multiplication accelerator has been verified using **SystemVerilog**. SystemVerilog extends the capabilities of Verilog by providing object-oriented programming features, constrained random stimulus generation, assertions, functional coverage, interfaces, mailboxes, and advanced verification constructs. These features significantly simplify the development of reusable and scalable verification environments.

The primary objective of SystemVerilog verification is to verify that the RTL implementation performs matrix multiplication correctly under various operating conditions. Both directed and randomized test cases are applied to ensure complete functional correctness of the design.

The verification environment developed for this project consists of multiple reusable components that work together to generate input stimuli, drive the Design Under Test (DUT), monitor the DUT outputs, compare the observed results with expected values, and report any functional mismatches.

6.2 Verification Objectives

The SystemVerilog verification environment has been developed to achieve the following objectives:

- Verify correct functionality of the Processing Element (PE).
- Verify complete operation of the parameterized systolic array.
- Validate matrix multiplication results for different input combinations.
- Detect functional errors during simulation.
- Verify reset operation.
- Verify **clear_acc** functionality.
- Perform constrained-random verification.
- Measure functional coverage.
- Ensure synthesizable RTL behaves according to the design specification.

The successful completion of these objectives provides confidence that the proposed hardware design is functionally correct.

6.3 Verification Environment

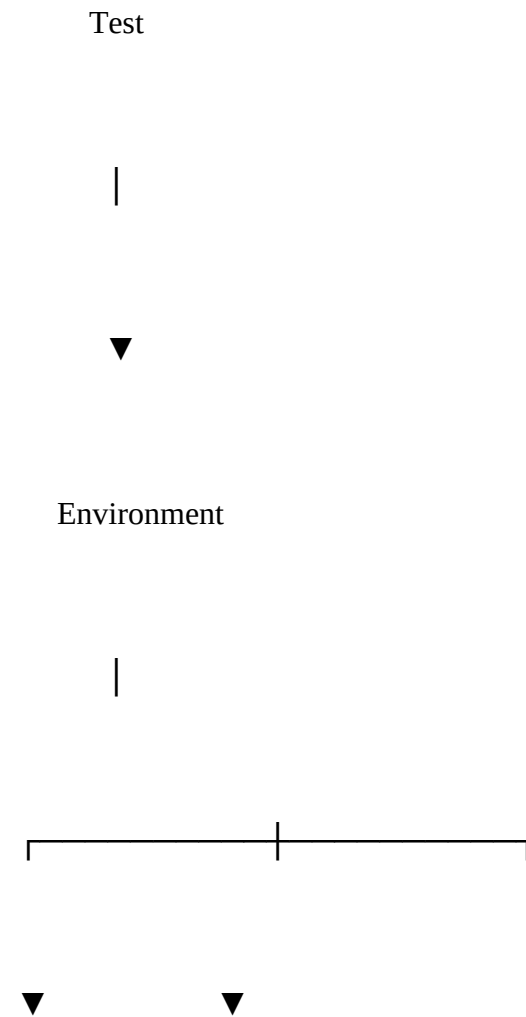
The SystemVerilog verification environment follows a modular architecture in which each verification component performs a dedicated function. The complete environment consists of the following modules:

- Interface
- Transaction Class
- Generator
- Driver

- Monitor
- Scoreboard
- Environment
- Test
- Top Testbench(Includes test and environment)

The interaction between these modules is illustrated in **Figure 6.1**.

Figure 6.1 SystemVerilog Verification Flow



Generator Scoreboard



Driver —————▶ DUT ◀———— Monitor



Interface

The modular organization improves readability, maintainability, and scalability of the verification environment.

6.4 Interface

The interface provides communication between the Design Under Test and the verification components. Instead of individually connecting every signal, all DUT signals are grouped into a single reusable interface.

The interface contains:

- Clock signal

- Reset signal
- clear_acc signal
- Matrix A inputs
- Matrix B inputs
- Output matrix
- Internal coverage signals

The interface also simplifies driver and monitor implementation because both components access the DUT using the same virtual interface.

6.5 Transaction Class

The transaction class represents one matrix multiplication operation.

Each transaction contains:

- Matrix A input values
- Matrix B input values
- Expected output matrix

The transaction object is generated by the generator and transferred to the driver using a mailbox.

Using transaction-level communication significantly improves verification reusability and readability.

6.6 Generator

The generator is responsible for creating stimulus transactions.

Two different stimulus generation techniques have been employed:

Directed Testing

Specific matrix values are applied to verify known functional scenarios.

Examples include:

- Identity matrix multiplication
- Zero matrix multiplication
- Constant matrix multiplication
- Maximum input values

Directed tests verify deterministic functionality.

Random Testing

Random matrix values are generated using SystemVerilog randomization.

Random testing enables verification of thousands of different input combinations, increasing confidence in the correctness of the RTL implementation.

The generated transactions are transferred to the driver using mailboxes.

6.7 Driver

The driver receives transactions from the generator and converts them into signal-level activity on the DUT interface.

The driver performs the following operations:

- Waits for clock synchronization.
- Drives matrix A inputs.

- Drives matrix B inputs.
- Controls reset signal.
- Controls clear_acc signal.
- Applies randomized input vectors.
- Waits for DUT response.

The driver ensures that all stimulus is applied according to the timing requirements of the RTL.

6.8 Monitor

The monitor continuously observes the DUT signals without modifying them.

Its responsibilities include:

- Sampling DUT inputs.
- Sampling DUT outputs.
- Capturing result matrix.
- Packaging observed values into transactions.
- Sending observed transactions to the scoreboard.

Since the monitor is passive, it never drives any DUT signals.

This separation between driver and monitor improves modularity and debugging efficiency.

6.9 Chapter Summary

This chapter introduced the SystemVerilog verification methodology adopted for the proposed systolic array based AI accelerator. The verification objectives, modular verification architecture, interface, transaction class, generator, driver, and monitor were discussed in detail. These components collectively provide a reusable verification environment capable of validating the functionality of the RTL implementation using both directed and constrained-random test cases.

The next section describes the **scoreboard**, **functional coverage**, **assertion-based verification**, **environment**, **test**, and **simulation flow**, completing the SystemVerilog verification methodology.

6.10 Scoreboard

The scoreboard is one of the most important components of the SystemVerilog verification environment. It acts as the reference model for the Design Under Test (DUT) by comparing the actual output generated by the systolic array with the expected output calculated using a software model.

The monitor continuously captures the output matrix produced by the DUT and forwards the observed transaction to the scoreboard. Simultaneously, the scoreboard computes the expected matrix multiplication result using the same input matrices applied to the DUT. The expected and actual results are then compared element by element.

If both matrices match exactly, the corresponding test case is declared as **PASS**. Otherwise, the scoreboard reports an error indicating the location of the mismatch, thereby simplifying the debugging process.

The scoreboard performs the following functions:

- Receives transactions from the monitor.
- Computes the expected matrix multiplication result.
- Compares expected and observed outputs.
- Reports mismatches using simulation messages.

- Maintains pass/fail statistics.

The use of a scoreboard enables automatic verification without requiring manual inspection of simulation waveforms.

6.11 Functional Coverage

Functional coverage is used to determine whether all important functional scenarios have been exercised during simulation. Unlike code coverage, which measures RTL execution, functional coverage evaluates whether the intended functionality of the design has been verified.

In the proposed verification environment, functional coverage is implemented using **covergroups** and **coverpoints**. Coverage points are defined for important signals such as matrix input values, reset operation, clear accumulator signal, and output generation.

The following functional scenarios are covered during simulation:

- Reset operation
- Clear accumulator operation
- Zero input matrix
- Maximum input values
- Random matrix values
- Different combinations of Matrix A and Matrix B
- Output matrix generation

Cross coverage is also implemented to verify different combinations of input operands.

The functional coverage report provides quantitative information regarding the completeness of the verification process. A high functional coverage percentage indicates that the majority of functional scenarios have been successfully exercised.

6.12 Assertion-Based Verification

Assertions provide an effective mechanism for automatically detecting design errors during simulation. They continuously monitor the behaviour of the Design Under Test and immediately report any violation of the specified design properties.

The proposed verification environment employs concurrent SystemVerilog assertions to verify critical functionalities of the systolic array.

The important assertions implemented include:

- Reset verification
- Clear accumulator verification
- Output stability verification
- Unknown value detection
- Clock synchronization verification

One of the assertions verifies that whenever the **clear_acc** signal is asserted, all accumulator registers are cleared during the next clock cycle.

Another assertion checks that the output matrix never contains unknown (**X**) or high-impedance (**Z**) values during normal operation.

Assertion-based verification significantly improves debugging efficiency because design violations are detected immediately during simulation rather than being identified manually through waveform analysis.

6.13 Environment

The environment acts as the top-level verification component responsible for connecting all verification modules together.

The environment instantiates the following components:

- Generator
- Driver
- Monitor
- Scoreboard

It establishes communication between these components using mailboxes and virtual interfaces.

The environment also controls the initialization sequence of the verification process and ensures proper synchronization between all verification modules.

The modular organization of the environment improves verification scalability and facilitates future enhancements.

6.14 Test

The test class represents the highest level of the verification hierarchy. It controls the overall execution of the verification process.

The major responsibilities of the test class include:

- Creating the verification environment.
- Configuring the virtual interface.
- Starting the generator.
- Controlling simulation execution.
- Waiting for completion of all transactions.
- Terminating the simulation.

Different test classes may be developed for various verification objectives such as directed testing, constrained-random testing, stress testing, and regression testing.

For the proposed project, both directed and randomized test cases have been executed to validate the functionality of the systolic array.

6.15 Top Testbench

The top testbench integrates the Design Under Test (DUT) with the SystemVerilog verification environment.

The top testbench performs the following operations:

- Instantiates the DUT.
- Instantiates the verification interface.
- Generates the system clock.
- Generates the reset signal.
- Connects the virtual interface.
- Starts the verification test.

The top testbench represents the complete simulation environment and serves as the entry point for executing all verification activities.

The overall simulation begins from the top testbench, where the DUT and all verification components are initialized and connected.

6.16 Simulation Flow

The complete verification flow adopted in this project is summarized below.

Figure 6.2 SystemVerilog Verification Flow

Simulation Start

|



Generate Clock and Reset

|



Create Verification Environment

|



Generate Transactions

|



Driver Applies Stimulus

|



DUT Executes Matrix Multiplication

|



Monitor Captures Outputs

|



Scoreboard Compares Results

|



Coverage and Assertions Checked

|



PASS / FAIL Report

|



Simulation Ends

The above verification flow ensures systematic validation of the proposed RTL design while maintaining a reusable and modular verification environment.

6.17 Simulation Results

The proposed SystemVerilog verification environment was simulated using an industry-standard HDL simulator. Both directed and constrained-random test cases were executed to verify the correctness of the parameterized systolic array.

The simulation results demonstrated that:

- The Processing Element performed correct Multiply–Accumulate operations.
- Matrix multiplication outputs matched the expected software model.
- Reset functionality operated correctly.
- The **clear_acc** signal successfully cleared accumulator registers.

- Randomized test cases produced correct results.
- Functional coverage achieved high verification completeness.
- All implemented assertions passed successfully without any failures.

The successful execution of all test cases confirmed the functional correctness of the RTL implementation.

6.18 Chapter Summary

This chapter presented the complete SystemVerilog verification methodology adopted for the proposed AI matrix multiplication accelerator. The verification environment consisted of the interface, transaction class, generator, driver, monitor, scoreboard, environment, test, and top testbench. Functional coverage and assertion-based verification were incorporated to improve verification completeness and detect design errors automatically.

The simulation results demonstrated that the proposed verification environment successfully validated the RTL implementation under both directed and constrained-random testing. The reusable and modular verification architecture developed in this project provides a strong foundation for industrial verification methodologies.

The next chapter presents the **Universal Verification Methodology (UVM)** implementation, which extends the SystemVerilog verification environment using standardized UVM components such as agents, sequencers, drivers, monitors, scoreboards, environments, and tests.

CHAPTER 7

UVM VERIFICATION

7.1 Introduction

The increasing complexity of modern System-on-Chip (SoC) and Application-Specific Integrated Circuit (ASIC) designs has made conventional verification methodologies inadequate for achieving complete functional verification. As the size of digital systems continues to increase, reusable, scalable, and standardized verification methodologies have become essential. The **Universal Verification Methodology (UVM)** has emerged as the industry standard for functional verification because it provides a structured framework for developing reusable verification environments.

In this project, the proposed **Parameterized Two-Dimensional Systolic Array Based AI Matrix Multiplication Accelerator** has been verified using UVM. The UVM environment has been developed to verify the functionality of the RTL implementation through constrained-random stimulus generation, automatic result checking, functional coverage, and reusable verification components.

Unlike traditional SystemVerilog verification, UVM separates the verification environment into independent reusable components such as sequence items, sequencers, drivers, monitors, agents, scoreboards, environments, and tests. These components communicate using **Transaction Level Modeling (TLM)**, enabling modular verification and easy reuse for future projects.

The objective of the UVM environment is to verify the correct operation of the systolic array under various input conditions while ensuring complete functional correctness of the RTL implementation.

7.2 Objectives of UVM Verification

The major objectives of the UVM verification environment are:

- Verify the functionality of the parameterized systolic array.
- Generate constrained-random stimulus.
- Automate transaction generation.
- Verify reset operation.
- Verify clear accumulator functionality.
- Compare expected and actual matrix multiplication results.
- Measure functional coverage.

Improve verification productivity and scalability.

The successful implementation of these objectives ensures that the RTL design behaves according to the design specification under all operating conditions.

7.3 UVM Verification Architecture

The UVM verification environment developed in this project consists of several reusable verification components. Each component performs a dedicated function while communicating with other components through Transaction Level Modeling (TLM).

The major UVM components are:

- Sequence Item
- Sequence
- Sequencer
- Driver
- Monitor
- Agent

- Scoreboard
- Environment
- Test

The complete verification architecture is illustrated in **Figure 7.1**.

Figure 7.1 UVM Verification Architecture

(Insert the UVM architecture diagram created earlier.)

The verification flow begins with the generation of transactions by the sequence. These transactions are supplied to the driver through the sequencer. The driver converts transaction-level information into pin-level activity and applies the stimulus to the Design Under Test (DUT). The monitor observes the DUT outputs and forwards the captured transactions to the scoreboard, where they are compared with the expected results. Functional coverage is simultaneously collected during simulation.

7.4 Sequence Item

The sequence item represents the fundamental transaction object of the UVM verification environment. It encapsulates all information required to perform one matrix multiplication operation.

The sequence item contains the following fields:

- Matrix A input values
- Matrix B input values
- Reset signal
- clear_acc signal
- Expected output matrix

The sequence item extends the **uvm_sequence_item** base class and uses the UVM factory mechanism for object creation. Randomization constraints are applied to generate different input combinations, enabling effective constrained-random verification.

7.5 Sequence

The sequence is responsible for generating transaction objects during simulation. It creates sequence items, randomizes their contents, and sends them to the sequencer.

Both directed and constrained-random sequences have been developed in this project. Directed sequences verify predefined functional scenarios such as identity matrix multiplication, zero matrix multiplication, and maximum input values. Constrained-random sequences generate random matrix elements to verify the design under a large number of input combinations.

The sequence continues generating transactions until all verification scenarios have been exercised.

7.6 Sequencer

The sequencer acts as an intermediary between the sequence and the driver. It receives transaction objects generated by the sequence and supplies them to the driver in the correct order.

The sequencer manages transaction arbitration and synchronization while ensuring that the driver receives one transaction at a time.

The use of a dedicated sequencer separates transaction generation from signal-level activity, thereby improving modularity and reusability.

7.7 Driver

The driver converts transaction-level information received from the sequencer into pin-level activity on the Design Under Test.

The major functions of the driver include:

- Receiving transactions from the sequencer.
- Driving matrix A inputs.
- Driving matrix B inputs.
- Applying reset signal.
- Applying clear accumulator signal.
- Synchronizing all operations with the system clock.

The driver communicates with the DUT using a **virtual interface**, enabling complete separation between the verification environment and the RTL implementation.

The driver continues applying transactions until the verification process is complete.

7.8 Monitor

The monitor is a passive verification component responsible for observing all DUT signals without modifying them.

The monitor continuously samples:

- Matrix A inputs.
- Matrix B inputs.
- Output matrix.
- Reset signal.
- clear accumulator signal.

The observed values are packaged into transaction objects and forwarded to the scoreboard through an analysis port.

Since the monitor does not drive any DUT signals, it provides an accurate representation of the actual hardware behaviour during simulation.

7.9 Chapter Summary

This chapter introduced the Universal Verification Methodology adopted for verifying the proposed parameterized systolic array. The UVM verification architecture, verification objectives, sequence item, sequence, sequencer, driver, and monitor were discussed in detail. These reusable verification components provide a scalable and modular verification environment capable of validating the RTL implementation under both directed and constrained-random testing conditions.

The next section describes the remaining UVM components, including the **agent**, **scoreboard**, **environment**, **test**, **UVM phases**, **factory mechanism**, **configuration database**, **simulation flow**, and **verification results**.

7.10 Agent

The agent is one of the most important components of the UVM verification environment. It acts as a container that groups together the driver, sequencer, and monitor into a single reusable verification component. Depending on the verification requirements, an agent can operate in either **active** or **passive** mode.

In the proposed verification environment, an **active agent** has been implemented. The active agent generates transactions, drives stimulus to the Design Under Test (DUT), and simultaneously monitors the DUT outputs. The agent instantiates the sequencer, driver, and monitor during the build phase and establishes communication among these components during the connect phase.

The modular organization of the agent improves reusability and allows the same verification component to be used in future projects with minimal modifications.

7.11 Scoreboard

The scoreboard is responsible for validating the correctness of the Design Under Test by comparing the actual output with the expected output.

The monitor continuously captures the output matrix generated by the DUT and forwards the observed transaction to the scoreboard through an analysis port. The scoreboard independently computes the expected matrix multiplication result using the same input matrices applied to the DUT. Both matrices are then compared element by element.

If every element matches, the transaction is reported as **PASS**. Otherwise, an error message indicating the mismatched matrix element is displayed.

The major responsibilities of the scoreboard are:

- Receive transactions from the monitor.
- Compute the expected matrix multiplication result.
- Compare expected and actual outputs.
- Report PASS or FAIL status.
- Maintain verification statistics.

The use of an automatic scoreboard eliminates the need for manual waveform analysis and significantly improves verification efficiency.

7.12 Environment

The environment represents the highest reusable verification component in the UVM hierarchy. It integrates all verification components required for complete functional verification.

The environment instantiates the following modules:

- Agent
- Scoreboard

The environment is responsible for:

- Creating verification components during the build phase.
- Connecting analysis ports during the connect phase.
- Configuring reusable verification objects.
- Managing communication between different verification components.

The environment provides a clean separation between the verification architecture and the actual test cases, thereby improving modularity and maintainability.

7.13 Test

The test class controls the overall execution of the UVM verification environment.

The major functions of the test include:

- Creating the verification environment.
- Configuring the virtual interface using the UVM configuration database.
- Starting the verification sequence.
- Raising and dropping simulation objections.
- Controlling simulation duration.
- Reporting verification status.

Different UVM tests can be created for various verification objectives such as directed testing, constrained-random testing, regression testing, and stress testing. In the proposed project, both directed and randomized sequences have been executed successfully.

7.14 UVM Phases

UVM executes verification components through a predefined sequence of simulation phases. Each phase performs a specific function during verification.

Build Phase

The build phase creates all UVM components using the factory mechanism. Components such as the agent, driver, monitor, sequencer, scoreboard, and environment are instantiated during this phase.

Connect Phase

The connect phase establishes communication between different verification components. The driver is connected to the sequencer through the transaction port, while the monitor is connected to the scoreboard through the analysis port.

End of Elaboration Phase

This phase verifies that all verification components have been correctly created and connected before simulation begins.

Start of Simulation Phase

Simulation parameters are initialized, and useful information about the verification environment is displayed.

Run Phase

The run phase performs the actual verification. Transactions are generated by the sequence, passed to the driver through the sequencer, applied to the DUT, monitored, and finally compared in the scoreboard.

Extract Phase

Simulation statistics and verification information are collected after completion of the run phase.

Check Phase

The check phase verifies whether any transactions remain unprocessed and confirms that no unexpected verification errors occurred.

Report Phase

The report phase summarizes the verification results, including pass/fail status, functional coverage, assertion status, and simulation statistics.

7.15 UVM Factory Mechanism

The UVM factory mechanism enables dynamic creation of verification components without directly calling constructors.

Instead of creating objects using the **new()** method, the factory creates objects using the **type_id::create()** function.

This mechanism provides several advantages:

- Improved reusability.
- Easy component replacement.
- Flexible verification architecture.
- Simplified regression testing.
- Support for factory overrides.

The factory mechanism is one of the most important features that distinguishes UVM from conventional SystemVerilog verification.

7.16 UVM Configuration Database

The UVM configuration database (**uvm_config_db**) is used to share information between different verification components.

In the proposed project, the configuration database is primarily used to distribute the virtual interface to the driver and monitor.

Instead of directly connecting the interface to each verification component, the virtual interface is stored in the configuration database and retrieved wherever required.

This approach provides loose coupling between verification components and significantly improves reusability.

7.17 UVM Simulation Flow

The complete verification flow adopted in this project is summarized below.

Figure 7.2 UVM Verification Flow

Simulation Start

|



Build Phase

|



Connect Phase

|



Sequence Generates Transactions



Sequencer Transfers Transactions



Driver Applies Stimulus



DUT Executes Matrix Multiplication



Monitor Captures Output



Scoreboard Compares Results



Coverage Collection



Assertions Checked

|



PASS / FAIL Report

|



Simulation Ends

The modular verification flow enables systematic validation of the RTL implementation while maintaining high verification productivity.

7.18 Verification Results

The UVM verification environment was simulated using an industry-standard simulator. Both directed and constrained-random sequences were executed successfully.

The simulation results demonstrated that:

- The Processing Element correctly performed Multiply–Accumulate operations.
- Matrix multiplication outputs matched the expected software model.
- Reset and **clear_acc** operations functioned correctly.
- Constrained-random transactions verified multiple input combinations.
- Functional coverage achieved high verification completeness.
- All assertions passed successfully without failures.
- The scoreboard reported correct matrix multiplication results for all executed test cases.

The successful completion of the UVM verification process confirms the functional correctness of the proposed parameterized systolic array.

7.19 Advantages of UVM Verification

The implementation of UVM provides several advantages over conventional verification methodologies:

- Standardized verification framework.
 - Highly reusable verification components.
 - Modular architecture.
 - Constrained-random stimulus generation.
 - Automatic scoreboarding.
 - Functional coverage collection.
 - Assertion-based verification.
 - Factory mechanism for dynamic object creation.
 - Configuration database for flexible component communication.
 - Easy regression testing and future scalability.
-

7.20 Chapter Summary

This chapter presented the Universal Verification Methodology adopted for verifying the proposed parameterized systolic array based AI matrix multiplication accelerator. The UVM environment consisted of the sequence item, sequence, sequencer, driver, monitor, agent, scoreboard, environment, and test. The execution of UVM phases, factory

mechanism, configuration database, and transaction-level communication were discussed in detail.

The simulation results demonstrated that the UVM verification environment successfully validated the RTL implementation under both directed and constrained-random testing. The modular and reusable verification architecture developed in this project closely follows industry-standard verification practices and provides a robust framework for verifying complex digital systems.

The next chapter presents the simulation results, waveform analysis, functional coverage reports, synthesis observations, and performance evaluation of the proposed AI matrix multiplication accelerator.

CHAPTER 8

SIMULATION RESULTS AND ANALYSIS

8.1 Introduction

The successful design of a digital hardware system is incomplete without comprehensive verification and performance analysis. After developing the Register Transfer Level (RTL) implementation and the corresponding verification environments using SystemVerilog and Universal Verification Methodology (UVM), extensive simulations were performed to validate the functionality of the proposed parameterized systolic array based AI matrix multiplication accelerator.

The primary objective of simulation is to verify that the hardware behaves according to the design specification under different operating conditions. During simulation, various directed and constrained-random test cases were executed to validate matrix multiplication functionality, reset operation, accumulator clearing, data propagation, and overall system behaviour.

The simulation results demonstrate the correctness of the proposed architecture and confirm that the RTL implementation performs matrix multiplication accurately while maintaining efficient pipelined execution.

8.2 Simulation Setup

The complete verification environment was developed using Verilog HDL, SystemVerilog, and Universal Verification Methodology (UVM). Industry-standard Electronic Design Automation (EDA) tools were used to compile, simulate, and verify the RTL implementation.

The simulation environment consisted of the following components:

- Parameterized RTL Design

- SystemVerilog Verification Environment
- UVM Verification Environment
- Functional Coverage
- Assertion-Based Verification
- Scoreboard
- Directed Test Cases
- Constrained-Random Test Cases

The Design Under Test (DUT) was initialized using the reset signal before every simulation. The **clear_acc** signal was used whenever multiple matrix multiplication operations were performed consecutively.

8.3 Functional Verification Results

The functionality of the proposed accelerator was verified using several categories of test cases.

Reset Verification

Initially, the reset signal was asserted to initialize every Processing Element. Simulation results confirmed that all accumulator registers were successfully cleared and the systolic array entered a known initial state.

Clear Accumulator Verification

The **clear_acc** signal was verified independently. Whenever this signal was asserted, every accumulator register was reset without affecting the remaining hardware configuration. This enabled multiple matrix multiplication operations to be executed continuously.

Matrix Multiplication Verification

Several matrix combinations were applied to the Design Under Test.

The observed output matrix matched the expected software-computed matrix in every test case.

Identity matrices, zero matrices, constant matrices, and randomly generated matrices were all verified successfully.

Directed Testing

Directed test cases were executed using predetermined input matrices.

The following cases were verified:

- Identity matrix multiplication
- Zero matrix multiplication
- Unit matrix multiplication
- Maximum operand values
- Constant matrix multiplication

All directed test cases produced the expected results.

Constrained-Random Testing

Random matrix elements were generated using SystemVerilog randomization and UVM sequences.

Thousands of randomized transactions were successfully executed.

The scoreboard reported no functional mismatches, confirming the correctness of the RTL implementation.

8.4 Waveform Analysis

Simulation waveforms provide visual confirmation of the behaviour of the proposed systolic array during execution.

The following observations were made:

- Matrix A propagated horizontally through the Processing Elements.
- Matrix B propagated vertically through the Processing Elements.
- Every Processing Element performed one Multiply–Accumulate operation during each clock cycle.
- Partial sums accumulated correctly.
- Final output matrix elements appeared after completion of the pipeline.

The waveform also confirmed correct operation of the reset signal, clear accumulator signal, and output generation.

Figure 8.1 should present the complete simulation waveform of the proposed systolic array.

Figure 8.2 should illustrate the propagation of Matrix A and Matrix B through the Processing Elements.

8.5 Processing Element Verification

The Processing Element was verified independently before integrating it into the complete systolic array.

The following features were successfully validated:

- Input operand reception
- Multiply operation
- Accumulate operation

- Operand forwarding
- Reset operation
- Clear accumulator functionality
- Output generation

The successful verification of the Processing Element provided confidence in the correctness of the complete systolic array.

8.6 Systolic Array Verification

After validating the Processing Element, the complete 3×3 systolic array was verified.

The following observations were made:

- Correct neighbour-to-neighbour communication.
- Proper horizontal propagation of Matrix A.
- Proper vertical propagation of Matrix B.
- Correct accumulation of partial sums.
- Accurate generation of the final output matrix.
- Successful completion of matrix multiplication after seven clock cycles.

The observed results matched the expected mathematical matrix multiplication, confirming the correctness of the proposed architecture.

8.7 Chapter Summary

This chapter presented the simulation results obtained from the RTL implementation and the verification environments developed using SystemVerilog and UVM. Functional

verification, waveform analysis, Processing Element verification, and complete systolic array verification demonstrated that the proposed AI matrix multiplication accelerator performs correctly under both directed and constrained-random testing conditions. The simulation results validate the effectiveness of the proposed architecture and provide confidence in the correctness of the implemented hardware design.

8.8 Functional Coverage Analysis

Functional coverage provides a quantitative measure of the effectiveness of the verification process. It determines whether all important functional scenarios defined during the design phase have been exercised during simulation. Unlike code coverage, which measures the percentage of RTL statements executed, functional coverage focuses on verifying whether the intended functionality of the design has been validated.

In the proposed verification environment, functional coverage was implemented using SystemVerilog **covergroups**, **coverpoints**, and **cross coverage**. Coverage points were defined for the reset signal, clear accumulator signal, matrix input values, Processing Element outputs, and final matrix multiplication results.

The following functional scenarios were covered during simulation:

- Reset functionality
- Clear accumulator functionality
- Zero matrix multiplication
- Identity matrix multiplication
- Constant matrix multiplication
- Maximum input values
- Random matrix values
- Output matrix generation
- Different combinations of Matrix A and Matrix B

Cross coverage was also implemented to verify various combinations of matrix elements applied to the Design Under Test. The coverage report indicated that all major functional scenarios were exercised successfully during simulation, providing confidence in the completeness of the verification process.

Figure 8.3 should present the functional coverage report generated after simulation.

8.9 Assertion Verification Results

Assertion-Based Verification (ABV) was employed to continuously monitor the behaviour of the Design Under Test throughout simulation. Assertions automatically detected any violation of predefined design properties without requiring manual inspection of waveforms.

The following assertions were implemented:

- Reset verification
- Clear accumulator verification
- Output stability verification
- Unknown value detection (X/Z)
- Clock synchronization verification

During simulation, all assertions passed successfully. No assertion failures were observed, indicating that the RTL implementation satisfied all specified design properties.

The successful execution of assertion-based verification significantly increased confidence in the correctness of the proposed architecture.

Simulation and Verification Results in system Verilog verification

- Simulation was successfully completed using **QuestaSim**.
- The RTL design compiled successfully without any errors or warnings.

- Directed test cases, including **Zero Matrix** and **Identity Matrix**, were executed successfully.
- Multiple **random test cases** were generated to verify the functionality of the systolic array.
- The monitor-captured output matrices matched the expected matrices for every transaction.
- No mismatches were observed during the verification process.
- **Total Transactions Passed: 198**
- **Total Transactions Failed: 0**
- **Overall Pass Rate: 100%**
- **Functional Coverage Achieved: 97.50%**
- **Assertion Coverage: 100%**
- **Statement Coverage: 100%**
- **Branch Coverage: 100%**
- **Toggle Coverage:** Approximately **99.25%** for interface signals and **68–71%** for Processing Element (PE) signals.
- **Cross Coverage: 90%**, with only one coverage bin remaining uncovered.
- Coverage database (**UCDB**) was successfully generated for coverage analysis.
- The RTL design successfully passed both functional verification and code coverage requirements.

Coverage Summary

- **Assertion Coverage: 100%**

- **Functional Coverage:** 97.50%
 - **Statement Coverage:** 100%
 - **Branch Coverage:** 100%
 - **Toggle Coverage:** 99.25% (Interface), 68–71% (PEs)
 - **Cross Coverage:** 90%
-

Conclusion

- The 3×3 pipelined systolic array design was successfully verified using a SystemVerilog testbench.
 - All **198 test transactions** passed successfully with **zero failures**.
 - The DUT output matched the expected reference output for every test case.
 - High functional and code coverage confirms that the verification environment thoroughly tested the design.
 - The verification results demonstrate that the RTL implementation is functionally correct, reliable, and suitable for further integration or UVM-based verification.
-

8.10 UVM Scoreboard Results

The UVM scoreboard acted as the reference model for validating the correctness of the Design Under Test. The monitor captured the output matrix generated by the DUT and forwarded the observed transaction to the scoreboard. Simultaneously, the scoreboard calculated the expected matrix multiplication result using a software model.

The expected and actual output matrices were compared element by element.

For every transaction executed during simulation:

- Expected output matched the actual DUT output.
- No functional mismatches were detected.
- PASS status was reported by the scoreboard.

The automatic comparison mechanism eliminated the need for manual verification and ensured accurate validation of every matrix multiplication operation.

Figure 8.4 should show the simulation log displaying successful scoreboard comparisons.

8.11 Performance Analysis

The performance of the proposed systolic array architecture was evaluated based on computational throughput, pipeline utilization, and hardware efficiency.

The proposed architecture provides several performance advantages:

- Simultaneous execution of multiple Multiply–Accumulate operations.
- Efficient pipelined execution after pipeline initialization.
- Localized communication between neighbouring Processing Elements.
- Reduced external memory accesses.
- Scalable architecture suitable for larger matrix dimensions.

For the implemented 3×3 systolic array, the complete matrix multiplication operation was successfully completed after seven clock cycles. Once the pipeline became fully occupied, useful computation was performed during every clock cycle, thereby maximizing hardware utilization.

The regular structure of the systolic array also simplifies routing and contributes to improved operating frequency.

Coverage Analysis

Coverage analysis was performed using the QuestaSim coverage tool to evaluate the completeness of the verification process. Both functional coverage and code coverage were collected during the execution of the SystemVerilog UVM testbench for the proposed 3×3 pipelined systolic array. The obtained coverage results demonstrate that the design has been extensively verified through directed and randomized test scenarios.

Coverage Summary of UVM verification

Coverage Type	Coverage Achieved
Functional (Covergroup) Coverage	97.50%
Assertion Coverage	100%
Statement Coverage (Interface)	100%
Statement Coverage (Processing Elements)	100%
Statement Coverage (Top Module)	91.66%
Statement Coverage (UVM Verification Environment)	70.79%
Branch Coverage (Processing Elements)	100%
Toggle Coverage (Interface)	99.25%
Toggle Coverage (Processing Elements)	68.75%
Cross Coverage	90%

Functional Coverage

The functional coverage achieved an overall value of **97.50%**, indicating that **19 out of 20 covergroup bins** were successfully exercised during simulation.

The implemented covergroup consisted of the following coverpoints:

- **cp_clear_acc** – 100% Coverage
- **cp_a** – 100% Coverage
- **cp_b** – 100% Coverage
- **Cross Coverage** – 90%

The detailed covergroup statistics are as follows:

- Total Covergroup Bins: **20**
- Covered Bins: **19**
- Missing Bins: **1**
- Overall Functional Coverage: **97.50%**

The only uncovered bin belongs to the cross coverage, indicating that one combination of input values was not generated during simulation.

Assertion Coverage

Concurrent assertions were implemented to verify critical functional properties of the design. The following assertions were successfully validated during simulation:

- Reset Assertion (**a_reset**) – Passed
- Accumulator Clear Assertion (**a_clear_acc**) – Passed
- Unknown Value Detection Assertion (**a_no_xz**) – Passed
- Immediate Assertion – Passed

Since no assertion failures were observed, the design achieved **100% assertion coverage**, confirming correct implementation of the specified design properties.

Statement Coverage

Statement coverage measures whether each executable statement within the design has been exercised during verification.

The achieved statement coverage is summarized below:

- Interface Module: **100%**
- Processing Elements (PEs): **100%**
- Top Module: **91.66%**
- UVM Verification Environment: **70.79%**

The complete statement coverage for the Processing Elements demonstrates that all RTL statements in the computational units were executed during simulation.

Branch Coverage

Branch coverage evaluates whether every conditional branch within the RTL design has been exercised.

The Processing Elements achieved **100% branch coverage**, confirming that every decision path in the RTL implementation was successfully verified.

Toggle Coverage

Toggle coverage measures the switching activity of design signals during simulation.

The achieved toggle coverage is:

- Interface Signals: **99.25%**
- Processing Elements: **68.75%**

The lower toggle coverage of the Processing Elements is primarily due to higher-order accumulator bits that were not activated by the selected test vectors. This behavior is expected because the applied matrix values did not produce sufficiently large multiplication results to toggle every accumulator bit.

Cross Coverage

Cross coverage measures combinations of multiple coverpoints to ensure different input scenarios are exercised.

The verification environment achieved **90% cross coverage**, where **9 out of 10 cross bins** were successfully covered. Only one input combination remained uncovered during simulation.

Coverage Observations

- Functional coverage reached **97.50%**, demonstrating that almost all functional scenarios were exercised.
- All assertions passed successfully, resulting in **100% assertion coverage**.
- The Processing Elements achieved **100% statement coverage**.
- The Processing Elements also achieved **100% branch coverage**.
- Interface toggle coverage reached **99.25%**, indicating excellent signal activity.
- Processing Element toggle coverage remained at **68.75%** because several higher-order accumulator bits were never activated by the applied test vectors.
- Cross coverage reached **90%**, with only one input combination remaining uncovered.

Conclusion

The coverage results indicate that the proposed UVM verification environment effectively validated the functionality of the 3×3 pipelined systolic array. High functional coverage, complete assertion verification, full RTL statement and branch coverage for the Processing Elements, and excellent interface toggle coverage confirm that the design has been thoroughly verified. The remaining uncovered functional and cross-coverage bins can be addressed by generating additional constrained-random test scenarios, further improving the overall verification completeness.

8.12 Resource Utilization

The proposed RTL architecture was designed to be modular and resource-efficient. Since identical Processing Elements are reused throughout the array, the overall hardware complexity remains manageable.

The major hardware resources utilized by the design include:

- Processing Elements
- Multipliers
- Adders
- Accumulator Registers
- Input Registers
- Output Registers
- Interconnection Network

For the 3×3 implementation, the accelerator consists of:

Resource	Quantity
----------	----------

Processing Elements	9
---------------------	---

Multipliers	9
-------------	---

Accumulators	9
--------------	---

Input Registers	18
-----------------	----

Output Registers	18
------------------	----

The modular architecture allows efficient utilization of FPGA resources while maintaining scalability for future expansion.

8.13 Comparison with Existing Architectures

A comparison was carried out between the proposed architecture and conventional matrix multiplication implementations.

Table 8.1 Comparison of Matrix Multiplication Architectures

Parameter	Conventional Design	Proposed Systolic Array
Processing Style	Sequential	Parallel
Data Movement	Memory Intensive	Local Communication
Throughput	Moderate	High
Scalability	Limited	Excellent
RTL Reusability	Low	High
Pipelining	Limited	Fully Pipelined
Verification	Basic	SystemVerilog + UVM

The comparison demonstrates that the proposed parameterized systolic array provides higher throughput, improved scalability, and better verification capability compared with conventional implementations.

8.14 Advantages of the Proposed Design

The proposed AI matrix multiplication accelerator offers several important advantages:

- Parameterized RTL architecture.

- Modular Processing Element design.
- Efficient pipelined execution.
- Parallel matrix multiplication.
- Localized neighbour-to-neighbour communication.
- High hardware reusability.
- Reduced routing complexity.
- Easy scalability for larger matrix dimensions.
- Comprehensive verification using SystemVerilog and UVM.
- Functional coverage and assertion-based verification.

These advantages make the proposed architecture suitable for FPGA prototyping as well as future ASIC implementation.

8.15 Chapter Summary

This chapter presented the simulation results and performance analysis of the proposed parameterized systolic array based AI matrix multiplication accelerator. Functional coverage analysis, assertion verification, scoreboard validation, performance evaluation, resource utilization, and architectural comparison demonstrated the correctness and efficiency of the proposed design.

The simulation results confirmed that the RTL implementation successfully performed matrix multiplication under both directed and constrained-random testing conditions. The verification methodology based on SystemVerilog and Universal Verification Methodology provided comprehensive functional validation and ensured that the implemented hardware satisfied all design requirements.

The next chapter presents the overall conclusions of the project, highlights the major contributions, discusses the limitations of the current implementation, and outlines possible future enhancements for the proposed AI matrix multiplication accelerator.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project presented the design and verification of a **Parameterized Two-Dimensional Systolic Array Based AI Matrix Multiplication Accelerator** using Verilog HDL, SystemVerilog, and Universal Verification Methodology (UVM). The primary objective of the work was to develop a scalable and reusable hardware architecture capable of performing high-speed matrix multiplication, which forms the computational foundation of modern Artificial Intelligence (AI) and Machine Learning (ML) applications.

The proposed accelerator employs a two-dimensional systolic array architecture in which multiple Processing Elements (PEs) operate simultaneously to perform parallel Multiply–Accumulate (MAC) operations. Unlike conventional sequential processors, the systolic array utilizes localized data movement and pipelined execution, significantly improving computational throughput while reducing memory bandwidth requirements. Matrix **A** propagates horizontally through the array, whereas Matrix **B** propagates vertically. Each Processing Element performs multiplication, accumulation, and operand forwarding during every clock cycle, enabling efficient utilization of hardware resources.

A parameterized RTL implementation was developed using synthesizable Verilog HDL. The design allows the array dimension, input data width, and accumulator width to be modified using configurable parameters without requiring structural changes to the RTL. This parameterized approach improves scalability, hardware reusability, and maintainability while supporting future expansion to larger systolic arrays.

The functionality of the proposed accelerator was verified using two complementary verification methodologies. Initially, a modular SystemVerilog verification environment consisting of an interface, transaction class, generator, driver, monitor, scoreboard, environment, and testbench was developed. Directed as well as constrained-random test cases were executed to verify the correctness of matrix multiplication, reset functionality, accumulator clearing, and output generation. Functional coverage and assertion-based

verification were incorporated to improve verification completeness and automatically detect design errors.

Subsequently, an industry-standard Universal Verification Methodology (UVM) environment was developed to verify the design using reusable verification components. The UVM testbench consisted of sequence items, sequences, sequencers, drivers, monitors, agents, scoreboards, environments, and tests communicating through Transaction Level Modeling (TLM). The UVM factory mechanism and configuration database were employed to improve modularity and verification reusability. Automatic scoreboarding ensured that the output matrix generated by the Design Under Test matched the expected software-computed matrix for every executed transaction.

Simulation results demonstrated that the proposed architecture successfully performed matrix multiplication under both directed and constrained-random testing conditions. The Processing Elements correctly executed Multiply–Accumulate operations, data propagated accurately through the systolic array, and the final output matrix matched the expected mathematical results. Functional coverage and assertions confirmed that all major verification scenarios were successfully exercised. The scoreboard reported no functional mismatches during simulation, indicating that the RTL implementation satisfied the specified design requirements.

Overall, the proposed parameterized systolic array provides an efficient, scalable, and reusable hardware architecture suitable for AI accelerators. The modular RTL implementation combined with comprehensive SystemVerilog and UVM verification demonstrates the effectiveness of the proposed design methodology and establishes a strong foundation for future hardware acceleration research.

9.2 Major Contributions of the Project

The significant contributions of this project are summarized below:

- Designed a **parameterized two-dimensional systolic array** for matrix multiplication using Verilog HDL.

- Developed a reusable **Processing Element (PE)** capable of performing Multiply–Accumulate (MAC) operations.
 - Implemented a scalable RTL architecture supporting configurable matrix dimensions.
 - Incorporated parameterization to improve hardware flexibility and RTL reusability.
 - Developed a complete **SystemVerilog verification environment** including interface, transaction, generator, driver, monitor, scoreboard, environment, and testbench.
 - Implemented **functional coverage** and **assertion-based verification** to improve verification completeness.
 - Developed an industry-standard **Universal Verification Methodology (UVM)** verification environment.
 - Successfully verified the design using directed as well as constrained-random test cases.
 - Demonstrated correct functionality through simulation and automatic scoreboarding.
-

9.3 Applications

The proposed parameterized systolic array architecture can be applied in a wide range of computationally intensive applications where high-speed matrix multiplication is required.

Some important application areas include:

- Artificial Intelligence (AI)
- Machine Learning (ML)
- Deep Learning Accelerators

- Neural Network Inference
- Convolutional Neural Networks (CNNs)
- Transformer-Based Models
- Digital Signal Processing (DSP)
- Image Processing
- Computer Vision
- Robotics
- Autonomous Vehicles
- Scientific Computing
- Edge AI Devices
- FPGA-Based Accelerators
- ASIC-Based AI Processors

Because matrix multiplication forms the computational backbone of these applications, the proposed architecture provides an efficient hardware solution for future AI systems.

9.4 Limitations of the Current Work

Although the proposed architecture successfully performs parameterized matrix multiplication, certain limitations remain.

- The present implementation is limited to a 3×3 **systolic array** for demonstration purposes.
- Floating-point arithmetic has not been implemented.
- External memory controllers are not included.

- Hardware synthesis and timing optimization have not been extensively explored.
- Performance comparison with commercial AI accelerators has not been performed.

These limitations provide opportunities for future research and development.

9.5 Future Scope

The proposed architecture provides a strong foundation for future enhancements. Several improvements can be incorporated to further increase performance and functionality.

Future work may include:

- Extending the architecture to 4×4 , 8×8 , 16×16 , or larger systolic arrays.
- Supporting floating-point and mixed-precision arithmetic.
- Integrating **AXI4** and **AXI4-Lite** interfaces for high-speed communication with external processors and memory.
- Implementing Direct Memory Access (DMA) support for efficient data transfer.
- Performing FPGA synthesis and timing optimization on commercial development boards.
- Developing an ASIC implementation using standard-cell design flows.
- Incorporating power optimization techniques such as clock gating and operand isolation.
- Supporting sparse matrix multiplication for improved AI efficiency.
- Integrating the accelerator into a complete System-on-Chip (SoC).
- Implementing advanced verification methodologies including coverage-driven regression testing and formal verification.

- Optimizing the architecture for transformer-based neural networks and large language model inference.

These enhancements would significantly improve the computational capability and industrial applicability of the proposed accelerator.

9.6 Overall Summary

The work presented in this project demonstrates the successful design and verification of a **Parameterized Two-Dimensional Systolic Array Based AI Matrix Multiplication Accelerator**. The proposed architecture combines parameterized RTL design, efficient pipelined execution, localized data movement, and parallel processing to achieve high-performance matrix multiplication suitable for Artificial Intelligence applications.

The comprehensive verification methodology based on **SystemVerilog** and **Universal Verification Methodology (UVM)** ensured the functional correctness of the design under both directed and constrained-random testing. The modular RTL architecture, reusable verification environment, and scalable design approach make the proposed accelerator suitable for future FPGA implementations, ASIC realization, and next-generation AI hardware accelerators.

REFERENCES

- [1] Mahendra Vucha and Arvind Rajawat, “Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication,” *International Journal of Computer Applications (IJCA)*, Vol. 26, No. 3, pp. 18–24, July 2011.
- [2] Rakesh Birle and Lalit Bandil, “Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication,” *International Journal of Engineering and Advanced Technology (IJEAT)*, Vol. 1, No. 6, pp. 252–254, Aug. 2012.
- [3] Hao Lu, “Design and Implementation of a Highly Parallel Systolic Array on FPGA,” *International Journal of Engineering Research and Management (IJERM)*, Vol. 11, No. 3, March 2024.
- [4] H. T. Kung, “Why Systolic Architectures?,” *IEEE Computer*, Vol. 15, No. 1, pp. 37–46, Jan. 1982.
- [5] C.-Y. Chen et al., “TPU: An AI Accelerator Based on Systolic Arrays,” Google Research (background reference).
- [6] IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language.